Uppsala University



Machine Learning

Assignment 5

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution.
- State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available in **the PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

```
1. install.packages("remotes")
```

2. `remotes::install_github("MansMeg/IntroML", subdir = "rpackage")`

- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You are not allowed to show your assignments (text or code) to anyone. Only discuss the assignments with your fellow students. The student that show their assignment to anyone else could also be considered to cheat. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Do not copy or upload the full assignment text into an LLM.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Contents

Automatic feedback on your R functions 4

Oral exam question contribution 5

1 Transformers and Attention 6

2 Implementing a simple RNN 8

3 *Recurrent Neural Networks with Keras 10

4 *BERT for Swedish with PyTorch 12

Automatic feedback on your R functions

The R package `markmyassignment` (MMA) provides automatic feedback on the functions listed in the selected check configuration. Define or load your own functions in R before running the checks. Use the `tasks` argument to check one function by name; omitting it checks every function in the selected configuration. Functions that depend on other functions need those definitions loaded too.

To select the checks for this assignment, run the following code in R:

```
install.packages("markmyassignment") # Install once
library(markmyassignment)
assignment_path <-
  paste("https://github.com/MansMeg/IntroML/",
        "blob/master/assignments/tests/assignment5/assignment5.yml", sep="")
set_assignment(assignment_path)
# List the function names available in this configuration
show_tasks()
# To check one function, replace function_name with its name:
# mark_my_assignment(tasks = "function_name")
# To check your code/functions, just run
mark_my_assignment()
```

The checks provide feedback on selected numerical cases; they do not assess the whole assignment or award marks. Print your complete function definitions in the PDF report as required by the general instructions, even when the checks pass. You do not need to include MMA results in the report. The supplied checks run R code; Python implementations are assessed from the report and cannot be run directly through MMA.

These checks cover the attention and RNN functions. The Keras and BERT experiments are assessed from your report.

Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 2–4 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

1 Transformers and Attention

Background reading: Follow the general reading instructions for Block 5.

We will implement a multi-head attention layer, a component of transformer-based models such as GPT and BERT. For a visual explanation, see Jay Alammar's The Illustrated Transformer, listed in the Block 5 reading instructions.

Text models map tokens to embeddings, combine context, and use the resulting representations to predict classes or tokens. Our attention layer builds contextual representations and is one component of a full transformer such as BERT. RNNs illustrate how the same learned update carries context through a sequence, and the VG tasks connect these representations to text classification.

Start by loading the parameters that we are going to use for the implementation as follows.

```
library(uuml)
data("transformer_example")
```

1. (1p) Start out by computing the query, key and value matrices for one attention head by implementing it in a function you call `qkv()`. The function should work like this.

```
# Pick out the matrix for the first attention head
Wq <- transformer_example$Wq[,,1]
Wk <- transformer_example$Wk[,,1]
Wv <- transformer_example$Wv[,,1]

# Pick out the first three words and their embeddings
X <- transformer_example$embeddings[1:3,]

res <- qkv(X, Wq, Wk, Wv)
res

## $Q
##             [,1]        [,2]       [,3]
## the    0.4722259  0.04995783 -0.5350845
## quick -0.3662435  0.12144160  0.3454785
## brown -0.1029677 -0.12728414  0.1817097
##
## $K
##                [,1]          [,2]       [,3]
## the    0.094360579 -0.203807092 -0.1851229
## quick -0.033313240  0.279012100  0.2530560
## brown -0.004457052  0.001013468  0.0133802
##
## $V
##                [,1]        [,2]        [,3]
## the    0.317318525 -0.35023010  0.13284078
## quick  0.009929565  0.04208206 -0.15412097
## brown -0.316413241  0.27717408  0.02725089
```

2. (2p) Now, based on your query, key and value, compute the attention for the three chosen tokens. Return a list containing the attention weights in `attention` and the attention output in `Z`:

$$A = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \qquad Z = AV.$$

Here d_k is the number of columns in K, and softmax is applied separately to each row. Each row of A sums to one. In the lecture notation, $\text{Attention}(Q, K, V)$ corresponds to Z.

```
attention(res$Q, res$K, res$V)
## $Z
##                [,1]          [,2]         [,3]
## the    0.012395453 -0.0212420459  0.009404870
## quick -0.003759269 -0.0008360029 -0.005108890
## brown  0.002412222 -0.0088974612  0.001147999
##
## $attention
##              the     quick     brown
## the   0.3601932 0.3080896 0.3317172
## quick 0.3088780 0.3582373 0.3328847
## brown 0.3300375 0.3360583 0.3339042
```

3. (1p) Interpret the attention weights. What does the second row mean?

4. (1p) How does the second row of attention weights determine the second row of Z?

5. (2p) Now we have everything in place for implementing it all as a multi-head attention layer. The layer will take in embeddings and then return a 3-dimensional embedding per word. First check your implementation using the three words below.

```
multi_head_self_attention(X,
                          transformer_example$Wq,
                          transformer_example$Wk,
                          transformer_example$Wv,
                          transformer_example$WO)
##                [,1]          [,2]         [,3]
## the   -0.014189613 -0.0040299008 -0.006756286
## quick -0.009963516 -0.0010724342 -0.001996524
## brown -0.006394562 -0.0006626115 -0.002219108
```

Then run it on all words using `X <- transformer_example$embeddings`. The full input produces nine output rows; its values will differ from the three-word example.

2 Implementing a simple RNN

We will implement a one-layer recurrent neural network (RNN) using `rnn_example`. Its construction and notation are adapted from Goodfellow et al. (2016, Section 10.2). Everything needed is specified below; no reading of that section is required. Reuse the same parameters at every time step.

Use four hidden units and three output units. Start from $h_0 = 0$, represented as a 4×1 column matrix. The inputs are the rows of `rnn_example$embeddings`.

```
library(uuml)
data("rnn_example")
hidden_dim <- 4
```

Hint: Each row of `X` contains one token embedding. Use `drop = FALSE` to preserve a single-row matrix. Within each RNN step, represent the hidden state as a 4×1 column matrix, and check that dimensions match before matrix multiplication.

1. (2p) Implement a simple RNN linear unit that takes the parameters `W`, `U`, `b`, the previous hidden state `h_t_minus_one`, and the input embedding `x_t`, and returns

   $$a_t = Wh_{t-1} + Ux_t + b.$$

   Here x_t is a column vector: transpose the single-row input embedding in R. With p input features and d hidden units, U is $d \times p$, W is $d \times d$, and b is $d \times 1$. The function should work as follows.

   ```
   X <- rnn_example$embeddings[1,,drop=FALSE]
   h_t_minus_one <- matrix(0, nrow = hidden_dim, ncol = 1)
   a_t <- rnn_unit(h_t_minus_one, X,
                   W = rnn_example$W,
                   U = rnn_example$U,
                   b = rnn_example$b)
   a_t
   ##
   ##              the
   ## [1,]  0.5819145
   ## [2,] -2.2686535
   ## [3,] -0.6410312
   ## [4,]  1.4891931
   ##
   ```

2. (1p) Implement the hidden-state update $h_t = \tanh(a_t)$, where the activation function $\tanh(a) = (e^a - e^{-a})/(e^a + e^{-a})$ is applied separately to each element.

   ```
   h_t <- activation(a_t)
   h_t
   ##
   ##              the
   ## [1,]  0.5240555
   ## [2,] -0.9788223
   ## [3,] -0.5656013
   ## [4,]  0.9031762
   ##
   ```

3. (1p) Implement the output scores $o_t = Vh_t + c$ and the softmax probabilities

$$\widehat{y}_{t,j} = \frac{\exp(o_{t,j} - m_t)}{\sum_{k=1}^{q} \exp(o_{t,k} - m_t)}, \qquad m_t = \max_k o_{t,k}.$$

With q output units, V is $q \times d$ and c is $q \times 1$. Subtracting the largest score gives the same probabilities while avoiding unnecessarily large exponentials. These functions should work as follows.

```
yhat_t <- softmax(output_rnn(h_t, rnn_example$V, rnn_example$c))
yhat_t
##
##             the
## [1,] 0.3063613
## [2,] 0.2930885
## [3,] 0.4005502
##
```

4. (2p) Now we are ready to implement the full recurrent layer. It should take an input matrix X and the neural network parameters and return the hidden states and the softmax output as follows.

```
X <- rnn_example$embeddings[1:3,,drop=FALSE]
rnn_layer(X,
          W = rnn_example$W,
          V = rnn_example$V,
          U = rnn_example$U,
          rnn_example$b,
          rnn_example$c)
##
## $ht
##               [,1]        [,2]       [,3]       [,4]
## the    0.52405551 -0.97882227 -0.5656013  0.9031762
## quick -0.05951368  0.03988226  0.8241800 -0.6562744
## brown -0.08984008  0.92822217 -0.1563247 -0.6657626
##
## $yhat
##             [,1]      [,2]      [,3]
## the   0.3063613 0.2930885 0.4005502
## quick 0.2838013 0.3490452 0.3671536
## brown 0.2878002 0.3666877 0.3455121
```

5. (1p) Now, what is the value of the hidden state `h_t` for the token `dog`?

6. (1p) Why would processing only `dog` give a different hidden state from processing the sentence up to `dog`?

3 *Recurrent Neural Networks with Keras

To get VG on this assignment, you can choose to do this task OR the task using BERT with PyTorch below. You are also free to do both, if you want. As long as you pass one of the VG assignments you will get a VG point. Although, please only write down the time it took to do the VG assignment for one of the tasks if you choose to do more than one.

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

Keras, TensorFlow, and Google Colab

This assignment uses `keras3` and `tensorflow` for R. Use the `keras3` package for all Keras code. Adam is the default optimizer: use `optimizer_adam(learning_rate = 0.001)` unless the task or supplied example specifies another learning rate or explicitly compares optimizers. To work in Google Colab, open the course R setup notebook, save a copy in Google Drive, and run its setup and training-check cells. Wait for "PASS: training and prediction work" before starting the assignment.

Use the assignment report template linked in the notebook to structure your report. Write explanations in text cells and put R code in code cells. Save your notebook and results in Drive, and run the setup cell again at the start of each R session.

For local installation, follow the course R setup guide. Details on the architecture components are available in the `keras3` reference.

We will use recurrent neural networks for text classification. Use the general reading instructions for Block 5 for conceptual background. Chapter 6–6.2 in Chollet and Allaire (2018) provides optional implementation help.

An RNN can produce an output at every time step or use only its final hidden state to produce one output for a complete sequence. Use the parameter notation from Section 2 when explaining your classifier below.

Start by downloading the IMDB dataset and preprocess it as follows.

```
library(tensorflow)
library(keras3)
maxlen <- 500
imdb <- dataset_imdb(num_words = 10000)
input_train <- pad_sequences(imdb$train$x, maxlen = maxlen)
y_train <- imdb$train$y
input_test <- pad_sequences(imdb$test$x, maxlen = maxlen)
y_test <- imdb$test$y
```

Runtime hint: Use the time taken by the first few epochs to estimate the remaining runtime. The first epoch may include extra setup overhead. These epochs count toward your existing ten-epoch run.

1. (2p) Create a simple ordinary neural network for text classification in Keras using an embedding layer of dimension 16 and a hidden layer with 32 hidden states (and an output layer with one unit). Use a validation split of 20% observations in the validation set and a batch size of 128. Train your network using Adam with learning rate 0.001 and use binary cross-entropy as the loss function. Print the model and return your validation accuracy after ten epochs. Reuse this data split and training setup for the RNN and LSTM baseline below.

2. (1p) Explain what the embedding layer is doing.

3. (1p) Now setup a simple Recurrent Neural Network with 32 hidden units and the same embedding dimension. Print the model and present your result.

4. (1p) Does your RNN classifier produce an output at every time step, or one output for the whole review? Identify which hidden state(s) feed the final output layer and explain your answer.

Example: RNN parameter counts *No submission required*

Consider a simple RNN with 3 input features, 2 hidden units and 1 output, with biases:

Parameters	**Dimensions**	**Count**
U: input-to-hidden weights	2×3	6
W: recurrent weights	2×2	4
Hidden biases	2	2
V: hidden-to-output weights	1×2	2
Output bias	1	1

Keras groups several mathematical parameters into a single layer's count. The `SimpleRNN` layer reports $6 + 4 + 2 = 12$ parameters; the `Dense` output layer reports $2 + 1 = 3$. Any embedding-layer parameters are counted separately.

5. (1p) Using the notation in Section 2, explain the roles of U and W. What are their dimensions in your classifier, and where are their parameter counts included in the Keras output?

6. (1p) Using the same notation, explain the role of V in your classifier. How many parameters does V contain, and where are they included in the Keras output?

7. (1p) Implement an LSTM baseline with 32 hidden units and `dropout = 0`, using the same embedding dimension and training setup as above. Print your model and validation accuracy.

8. (1p) Starting from your LSTM baseline, design and test two modifications of your choice. Before training, explain what you expect each modification to change and why. Change one aspect at a time, keeping the data split and other settings fixed. Include the model summaries and a compact table of validation accuracy and training time for all three models. Discuss whether the results support your expectations. Improved accuracy is not required for full credit.

9. (1p) Using training and validation performance, discuss evidence of underfitting or overfitting. Can these results indicate the Bayes error? Explain.

4 *BERT for Swedish with PyTorch

To get VG on this assignment, you can choose to do this task OR the task using Keras and PyTorch above. You are also free to do both if you want. You will get a VG point if you pass one of the VG assignments. Although, please only write down the time it took to do the VG assignment for one of the tasks if you choose to do more than one.

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

Unfortunately, there is yet to be a simple way to run BERT directly from R. Although, using Google Colab, we can run one of the Swedish BERT models trained by the Swedish national library. The tutorial is in English, and without any Swedish understanding, it should be straightforward to follow.

Getting started: Open the Python notebook in Google Colab. Run the supplied setup, data-loading and preprocessing cells unchanged, then run the provided model to obtain your baseline. Use the model and training sections to implement your chosen experiments.

1. (2p) Design and run motivated experiments to improve the BERT baseline. Aim for 80% overall accuracy. Well-motivated experiments can receive full credit even if this target is not reached. Some possible changes are:

   (a) Fine-tune a larger part of the BERT model (not just the classification head)

   (b) Run the model for more epochs

   (c) Try out other learning rates (scheduling)

   Note! Training these large neural networks might take some time. It can take more than an hour at Google Colab to fine-tune the whole BERT model.

2. (1p) Report the baseline overall accuracy, exactly what you changed, and the resulting overall accuracy. Also report your final model's accuracy per party.

3. (2p) Reason based on your final confusion matrix on the difficulties for the model. Can the errors be reduced further or not? Why?

References

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

François Chollet and Joseph J Allaire. *Deep Learning with R*. Manning, 2018.